

TERNARY COMPUTERS⁺

PART 1: MOTIVATION FOR TERNARY COMPUTERS

By

G. Frieder

PART 2: EMULATION OF A TERNARY COMPUTER

By

G. Frieder and C. Luk

State University of New York at Buffalo
Computer Science, Amherst, New York

Abstract

In the first part, motivation is presented for a study of Ternary Computers using emulation. Areas of interest and goals of evaluation are briefly outlined.

In the second part, a representation of balanced ternary numbers is presented and examined. Microcode is then written to implement parallel operation on these ternary numbers. Results of experiments are reported in order to demonstrate that the goals outlined in Part 1 can be achieved via emulation.

Part 1 is based on Project MU report 32-72-MU. Part 2 is based on Project MU report 34-72-MU. These reports are not reproduced here in full for sake of brevity.

Part 1 - Motivation

Introduction

It is an interesting and worthwhile experience to look through the literature of the early 50's and to trace the development of "High Speed Computing Devices". The state of the art of that period is possibly best summarized in (1). Very roughly, one can distinguish, in that time, a noted effort to find answers to questions such as what kind of arithmetic is to be used and what kind of hardware representations are the proper ones to be utilized in computers. The main and most important considerations of that period were component counts and device reliability. The binary computer was not a fait accompli, and logics other than binary were considered.

That period of computer pioneering brought forth the dominance of the binary computer, based, as mentioned before, on analysis of hardware

properties. Of major importance to our discussion here is the fact that designers and researchers of that time lacked widely accessible and easily used simulation devices and techniques. The binary componentry and logic were not a result of thorough analysis of possible applications. One also recalls that, at that time, the use of computers was centered mainly on computational problems. That could limit any possible analysis, even if it would be done, to arithmetical structures only.

We shall refer to those times as the "first period".

In latter days, referred to here as "the second period", the main emphasis was on the architecture of computer systems.

This period, centered in the beginning of the 60's, used the concepts of a binary computer in a curiously sacred way. It seemed that the approach was "In the Beginning, Man Created Binary Computers". The basics of computer arithmetic were not questioned any longer, and the basic question was how to put it all together. There is one exception to this state of affairs, and that is the support of decimal arithmetic in hardware or, what is equivalent for the user, more recently in firmware. This has to be taken as a concession made by "binary" designers to the fact that in the users' domain, decimal arithmetic is highly desirable. It is, however, supported on binary rather than decimal logic, and serves only arithmetic, and not logical, purposes. The second period, by the way, was the period of rapid development of sophisticated simulation.

We are thus faced today with computers in which the basic concepts were checked out and evaluated by incomplete criteria, i.e. those of hardware implementation. This is the reason that we have the choice between

⁺Work supported by NSF Grant #GJ-993.

various internal data representations which are lacking in one aspect or another. For example, the binary representation of negative numbers by one's complement is very attractive, as it is connected naturally to the logical bitwise NOT operation. On the other hand, every red-blooded mathematician shudders when presented with positive and negative zeros. Two's complement notation, which has only one zero, is available, but its connection to logical operations is inconvenient to say the least.

The fact that binary computers eventually dominated the commercial field has not caused people to stop considering non-binary logic, and considerable work was done in that area (2,3). The work done was both in the theoretical and the practical aspects and may be summarized as follows:

In the theoretical aspects there is a sound foundation for a multivalued logic design. The switching algebra is well understood and its relation to Boolean algebra can be demonstrated.

In the practical aspects there is an emergence of multistable devices ranging from an n-flop (a device with n stable and distinguishable states) to complete arithmetic and logical units (see, for example, reference 3).

One conclusion which is very evident from the available literature is that we are rapidly approaching an era in which the dominance of binary hardware may not exist anymore. Thus the time seems right to inquire into the question of the architecture and function of the computers which the various multivalued logic technologies may prove possible.

The possible emergence of the multivalued logic computers prompts us to try to avoid the mistakes which were made in the first period of computer development. We believe that with the tools we have today, one should start in a different way than in the past. We suggest that instead of developing hardware units and forcing on the user community those features which are, in the eyes of the engineers, the most feasible, one should look first for the possible ways of using the hardware. What it all amounts to is that instead of the historical bottom up approach, i.e., structuring computers from given hardware units, we should take the top-down approach. This means that we have to start with the various applications in mind and ask what kind of units do we need in the computer in order to accomplish the users' goals.

Now, statements like this are easy to issue, but difficult to implement. Given a problem, what do you do in order to learn what is the impact of the solution of that problem on the structure of the computer? And even if you find out, how do you know that other problems will not suggest a different computer structure?

The solution, as we propose it, is to use the highly sophisticated simulation tool available today - the general purpose microprogrammable computer. We suggest to implement the top-down approach by emulating, on present hardware, complete multivalued machines, with the most flexible and least restrained arithmetic and logical structures. No apriori decisions should be made relative to representation of numbers, restriction of logical operations, etc.

In the following sections we shall introduce a ternary computer, describe its architecture, examine its execution units in some detail, and then state our evaluation goals. The second part of the presentation will introduce our preliminary results.

Ternary Computers

Among all possible multivalued logics, ternary seem to be the simplest and, therefore, the most suitable as a starting point. It was once considered seriously as a replacement for binary logic (1) and rejected mainly on the basis of particular hardware considerations which are no longer valid. There seems to be quite an abundance of hardware units built for ternary computers, including adders, multipliers, etc. (3). Furthermore, a ternary computer was built in the University of Moscow in the late 50's (9). These existing devices illustrate exactly the problem to which we referred in the previous passage; they employ base-3 arithmetic without any reference as to how such arithmetic should be applied. Although ternary systems were known for a while, and in particular balanced ternary was looked into (4), there is, to the best of my knowledge, no apriori attempt in those systems to decide if arithmetic should be implemented in balanced or regular ternary.

A similar argument can be applied to the logical unit. The number of basic ternary connectives is very large (3^{**9}) and one has to decide which one of those is of interest. Here, as well, there is no indication in the literature of any study as to which connectives have to be supplied.

We propose to "build" a ternary computer, as described in the following sections, in order to explore these and other questions.

Machine Architecture

In order to facilitate easy software development, we propose to implement a rather conservative architecture. Our machine, the TERNAC, will be a register machine very similar to the multiregister machines available today.

The basic features of the machine, which are described in greater detail later, are summarized in the following (trit stands for ternary digit):

3**8=6561 words of storage, each 18 trits wide,

Balanced ternary arithmetic unit (i.e. digit representation -1, 0, 1),

Regular ternary mode (i.e. digit representation 0, 1, 2),

Ternary logical unit with Boolean subset,

Shifter (of ternary data) for logical and circular shifts,

5 registers for fixed point and trit-wise logical operations, 18 trits wide,

5 registers for floating point and double precision operations, 36 trits wide.

For the sake of simplicity, all operations are register to register. There will obviously be load-store operations which will transfer single or multiple length quantities between the registers and memory.

The register-to-register operations involve, except for the operation itself, the ability to sense the status of the result, perform a skip, dependent on the status, and/or inhibit the storage of the result, conditionally or unconditionally.

It is evident that this machine basically differs from a usual binary computer in three areas: the basic unit of information is a trit; the arithmetic unit can operate in different modes; and the logical unit, while retaining all Boolean connectives, has a third logical state.

The I/O instructions are concerned only with character string input and output. All conversions will be done by programs resident in the ternary machine.

The number of registers was chosen rather arbitrarily. It does consist of an emulation compromise, but it can be changed very easily. The structure of the machine instruction will allow a far

higher number of registers, possibly as high as 27.

One final remark on the machine architecture is in order here. It is not the intention of this writer to advocate multi-register machines as the best architecture. They are very simple to construct in microprogramming and are simple to use. In the framework of Project MU at SUNY/AB, other machine architectures are investigated. If the ternary machine will prove to be worthwhile, we shall possibly implement a far richer and more elaborate machine using emulated ternary units.

Evaluation Goals

The ternary machine, as presented, is not different in architecture from the common computers used today. The difference lies in the type of arithmetic and types of logical capabilities of that machine. The value of the machine is thus not in evaluation of architecture, but in evaluation of the various algorithms that non-binary machines can naturally support. These algorithms should be taken from the following areas:

1. Numerical Analysis: Round-off sensitive algorithms, for instance, should be screened very carefully. The nature of balanced ternary arithmetic indicates that it may be superior, in that area, to both binary and non-balanced ternary (4).

2. Decision table processing and information retrieval: In these areas the ability to define three logical states, i.e., "True", "False" and "Don't care" or "Non-defined" should pose intriguing possibilities (7, item 3).

3. Non-deterministic or heuristic procedures: The author confesses complete ignorance of these areas, yet it seems that a third logic state can be of value here.

4. Logical problems in design: The abundance of literature in this area in recent years (see 7) is impressive. The fact that people had to implement logical problems which are essentially non-binary in nature, using binary computers, is an unnecessary handicap.

5. Higher-level languages: Although the impact of binary computers on higher-level languages has not been evaluated in any depth, non-binary logic should have some impact in this area as well.

Each of the areas mentioned will be carefully screened, and experiments will be conducted to find if the ternary

machine has an advantage over similar binary machines. The logic connectives used, as well as other instructions, will be monitored and their usefulness evaluated.

The way in which the machine is implemented, i.e. in Version 1.2 of a level-1 machine on top of QM-1 (5) dictates the procedures of evaluation. As there was no attempt to utilize in any optimal way the binary hardware on which we are emulating, there is no point whatsoever to do any speed comparisons between the ternary and binary computers. The final evaluation will consist of monitoring the types of operations performed, the number of instructions used in each category, memory utilization, etc. These numbers will then be compared to binary machines performing the same type of work. Relations between these two sets of data will be used for evaluation purposes.

We now refer the reader to Part II where the results of our preliminary work are reported.

Part II - Emulation

INTRODUCTION

As the first part of a complete emulation of a ternary computer, the authors undertook the study of the emulation of the balanced ternary arithmetic unit.

Briefly, a balanced ternary number is a string of symbols a of the form

$$a_m a_{m-1} \dots a_1 a_0 \cdot a_{-1} a_{-2} \dots a_{-k}$$

This string represents a number

$$\sum_{i=-k}^m a_i \cdot 3^i$$

where $a_i \in [-1, 0, 1]$, and, as pointed out earlier, a_i are referred to as trits. The numbers represented are either positive or negative. The sign is determined by the leading nonzero trit.

In what follows we shall describe the representation of trits on our binary hardware, and our implementation of a fixed point balanced ternary unit. The implementation goals were:

1) Easy representation of trits on binary hardware

2) Suitability of that representation for logical and non-balanced operations.

3) Parallel operations on trits in at least addition or subtraction.

4) Adequate speed. By that we mean speed which will enable us to use this unit in a computer and still be able to run meaningful experiments.

5) Implementation in a dynamically changeable microprogrammed environment.

The first goal is dictated by our usage of binary hardware. The second point reflects our wish to be able to implement, eventually, a complete ternary machine. The third and fourth are really connected. The former reflects the fact that serial operations, as will be pointed out in the conclusions, are too slow. The latter is triggered by our wish to have a usable system. Rather arbitrarily, we defined "adequate speed" to be not less than the speed of IBM 360/30. Point five reflects our desire to be able to change and adopt the arithmetic unit to reflect our hopefully growing understanding of ternary processes.

Representation:

The following notations will be used:

X_t, A_t, B_t	Balanced ternary numbers
'+'	Balanced ternary addition operator
X, X_1, X_2, A_1, A_2	Binary numbers

The balanced ternary number system, as presented in the introduction, is a base 3 system using the numbers -1, 0, and +1. We shall use T to denote -1 (T is chosen to denote -1 because it looks nice on computer output, resembling a 1 with a bar on top). To facilitate future discussions, all balanced ternary numbers will be suffixed with the letter "t", e.g. $1t, 0t, Tt, Xt, At, Bt$, where Xt, At and Bt are symbolic representations of balanced ternary numbers. Note that Tt has the special meaning of $-1t$. A number, or the symbolic representation of a number, that does not have the suffix "t" will be a binary number. The following are some examples:

Balanced ternary: $1t, 0t, Tt, 1101T1t, TT01T1t, At, Bt, Xt$.

Binary: $1, 0, 10111, A, B, X, A_1, A_2, B_1, B_2, X_1, X_2, LB$.